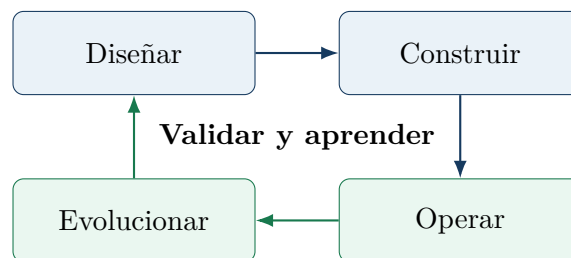


Especialización en Ingeniería de Software

Arquitectura de Software Moderna

y desarrollo asistido por inteligencia artificial

Libro de Informática I



Preparado por los profesores

Dr. Carlos Montenegro Marin

Dr. Esteban Hernández Barragán

Enfoque práctico con herramientas de software libre

Bogotá, 2026

Presentación

Este libro desarrolla el syllabus actualizado de *Informática I*. Su propósito no es convertir al lector en operador de un catálogo de herramientas, sino formar criterio para tomar decisiones arquitectónicas, producir evidencia verificable y asumir responsabilidad por sistemas que deben funcionar en condiciones reales.

Los capítulos utilizan un caso conductor llamado **LibreReserva**: una plataforma para reservar espacios y recursos compartidos de una institución educativa. El caso comienza como un monolito modular, incorpora una API, persistencia, mensajería, automatización, observabilidad, seguridad y, finalmente, asistencia de inteligencia artificial (IA). La evolución permite comparar alternativas sin presentar los microservicios, la nube o la IA como soluciones universales.

Las herramientas privilegiadas son de software libre o de código abierto. Algunas imágenes de contenedor y modelos de IA poseen licencias propias; por ello, antes de usarlos en un proyecto real se debe revisar la licencia concreta, la política institucional y las restricciones sobre datos.

Uso responsable de IA

La IA puede apoyar análisis, generación de código, pruebas, refactorización y documentación. Toda contribución generada debe ser declarada, revisada, probada y comprendida. El estudiante conserva la autoría académica y la responsabilidad técnica. No deben enviarse credenciales, datos personales, código confidencial o propiedad institucional a servicios no autorizados.

Cómo usar el libro

Cada capítulo contiene cuatro elementos recurrentes:

- **Explicación:** fundamentos, decisiones y límites.
- **Representación:** diagramas y modelos que hacen explícito el razonamiento.
- **Laboratorio:** práctica acumulativa sobre LibreReserva.
- **Validación:** comandos, pruebas y criterios de aceptación que permiten demostrar el resultado.

Los comandos se muestran para GNU/Linux y macOS. En Windows se recomienda WSL2 para mantener un entorno compatible. No ejecute comandos de instalación con privilegios administrativos sin revisar antes su procedencia y efecto.

Índice general

Presentación	i
1. Sistemas modernos y entorno reproducible	1
1.1. La arquitectura como conjunto de decisiones	1
1.2. Ciclo de trabajo basado en evidencia	2
1.3. Software libre y reproducibilidad	2
1.4. Preparación del laboratorio	3
1.4.1. Repositorio inicial	3
1.5. Ejercicios	4
1.6. Lista de comprobación	4
2. Requisitos, restricciones y atributos de calidad	5
2.1. Del deseo a la condición comprobable	5
2.2. Atributos relevantes	5
2.2.1. Desempeño y escalabilidad	5
2.2.2. Disponibilidad y resiliencia	6
2.2.3. Seguridad y privacidad	6
2.2.4. Modificabilidad y operabilidad	6
2.3. Restricciones y decisiones	6
2.4. Funciones de aptitud	6
3. Comunicación de arquitectura: C4, UML y ADR	8
3.1. Una vista responde una pregunta	8
3.2. Contexto y contenedores de LibreReserva	9
3.3. Cuándo emplear UML	9
3.4. ADR: decisiones que sobreviven a la reunión	9
3.5. Diagramas como código	10
3.6. Ejercicios	10
4. Modularidad, dominio y patrones	12
4.1. Modularidad antes que distribución	12

4.2. Lenguaje ubicuo y límites de dominio	12
4.3. Arquitectura hexagonal	13
4.4. Patrones y límites	13
4.4.1. MVC, Repository y DAO	13
4.4.2. Inversión de dependencias	13
4.5. Esqueleto modular en Python	14
4.6. Ejercicios	15
5. Estilos arquitectónicos y sistemas distribuidos	16
5.1. Los estilos son restricciones deliberadas	16
5.2. Monolito modular	16
5.3. Microservicios	16
5.4. Eventos y mensajería	17
5.5. Serverless	17
5.6. Árbol de decisión	17
5.7. Patrones de evolución	17
6. APIs, datos, consistencia y mensajería	19
6.1. API como contrato	19
6.2. Idempotencia y concurrencia	19
6.3. Persistencia y propiedad de datos	19
6.4. Mensajería asíncrona	20
6.5. Plataforma local	20
6.6. Ejercicios	21
7. Implementación evolutiva y pruebas automatizadas	22
7.1. Construir por incrementos verificables	22
7.2. Una estrategia de pruebas por riesgo	22
7.3. Dobles, propiedades y datos de prueba	23
7.4. Ejemplo: contrato HTTP comprobable	23
7.5. Cobertura, mutación y calidad	24
7.6. Preguntas de revisión	25
8. Entrega continua, contenedores e infraestructura reproducible	26
8.1. Del cambio al servicio operable	26
8.2. Git como protocolo de colaboración	26
8.3. Contenedores sin demonio privilegiado	27
8.4. Integración continua con Forgejo Actions	27
8.5. Despliegue local con Kubernetes	28

8.6. Configuración, secretos e infraestructura como código	29
8.7. Preguntas de revisión	29
9. Observabilidad, confiabilidad y costo	30
9.1. Observar para explicar	30
9.2. Diseño de señales	30
9.3. SLI, SLO y presupuesto de error	31
9.4. Instrumentación libre y neutral	31
9.5. Validar con una hipótesis operativa	32
9.6. Rendimiento, resiliencia y costo	32
9.7. Preguntas de revisión	32
10.Seguridad, privacidad y cadena de suministro	33
10.1. Seguridad como propiedad del sistema	33
10.2. Modelado de amenazas	33
10.3. Identidad, autorización y secretos	34
10.4. Privacidad y abuso	34
10.5. Dependencias y procedencia	35
10.6. Respuesta y recuperación	35
10.7. Preguntas de revisión	36
11.Desarrollo asistido por inteligencia artificial	37
11.1. Qué cambia y qué permanece	37
11.2. Anatomía de una solicitud útil	37
11.3. Usos y controles	38
11.4. Agentes y permisos	38
11.5. Modelos locales y herramientas libres	38
11.6. Protocolo de evaluación	39
11.7. Riesgos y gobernanza	39
11.8. Preguntas de revisión	40
12.Proyecto integrador: LibreReserva operable	41
12.1. Problema y restricciones	41
12.2. Resultados de aprendizaje y evidencias	41
12.3. Iteraciones	42
12.4. Definición de terminado	42
12.5. Rúbrica	43
12.6. Validación final	43
12.7. Defensa	43

12.8. Cierre	44
A. Guía de instalación y entorno reproducible	45
A.1. Capas del entorno	45
A.2. Perfil mínimo y perfil ampliado	46
A.3. GNU/Linux Debian o Ubuntu	46
A.4. macOS	46
A.5. Windows con WSL2	46
A.6. Entorno Python reproducible	47
A.7. Comprobación integral	47
A.8. Diagnóstico seguro	47
B. Plantillas de trabajo y evaluación	49
B.1. Registro de decisión arquitectónica	49
B.2. Escenario de atributo de calidad	50
B.3. Registro de amenaza	50
B.4. Registro de uso de IA	51
B.5. Informe de experimento	51
B.6. Lista de revisión de código	51
B.7. Paquete de evidencia	52
Referencias y recursos técnicos	53

Índice de figuras

1.1. Ciclo de decisión arquitectónica guiado por evidencia.	2
2.1. Escenario medible de desempeño para LibreReserva.	5
3.1. Niveles de abstracción del modelo C4.	8
4.1. El dominio depende de puertos; los adaptadores dependen del dominio.	13
5.1. Decisión conservadora antes de distribuir.	17
6.1. Patrón outbox para publicación confiable.	20
7.1. Ciclo corto de implementación y validación.	22
8.1. Cadena mínima de evidencia de entrega.	26
9.1. Flujo conceptual de telemetría; los backends concretos son sustituibles.	30
10.1. Flujo de datos simplificado y límite de confianza.	34
11.1. Bucle de asistencia con decisión y evidencia humanas.	37
12.1. Arquitectura de referencia, deliberadamente sustituible mediante ADR.	41

Índice de cuadros

2.1. Ejemplo de comparación de alternativas.	6
7.1. Selección de pruebas según el riesgo.	23
9.1. Especificación de un SLI útil.	31
10.1. Extracto del modelo de amenazas.	34
11.1. Usos de IA y evidencia mínima.	38
12.1. Matriz de trazabilidad del proyecto.	42
12.2. Rúbrica analítica del proyecto final.	43
A.1. Herramientas por capacidad.	45

Capítulo 1

Sistemas modernos y entorno reproducible

Objetivos del capítulo

Al terminar este capítulo, el lector podrá explicar qué es una arquitectura de software, distinguir decisiones de arquitectura de decisiones locales de implementación, preparar un entorno reproducible y registrar evidencia técnica desde el primer cambio.

1.1. La arquitectura como conjunto de decisiones

La arquitectura de software describe las estructuras significativas de un sistema, las responsabilidades que contienen, sus relaciones y las razones que justifican su forma. No se reduce al diagrama inicial ni al framework utilizado. Una arquitectura existe aunque nadie la haya documentado: emerge de dependencias, límites de despliegue, modelos de datos, decisiones de seguridad y mecanismos de operación.

Una decisión es arquitectónicamente significativa cuando cambiarla tarde resulta costoso o cuando condiciona atributos como disponibilidad, desempeño, seguridad, mantenibilidad o costo. Elegir el nombre de una variable suele ser local; decidir que cada módulo posea su propio esquema de datos modifica el acoplamiento, la consistencia y la operación.

La arquitectura se evalúa en contexto. Una solución apropiada para un prototipo de veinte usuarios puede ser irresponsable para un sistema clínico. Por ello, la primera pregunta no es “¿qué tecnología está de moda?”, sino “¿qué problema, restricciones y consecuencias debemos manejar?”.

Caso conductor: LibreReserva

LibreReserva permite que estudiantes y docentes consulten disponibilidad y reserven salas, laboratorios o equipos. El sistema debe evitar reservas simultáneas, conservar auditoría, proteger datos personales y continuar operando durante periodos de alta demanda. En la primera iteración habrá un único equipo de desarrollo y un despliegue institucional.

1.2. Ciclo de trabajo basado en evidencia

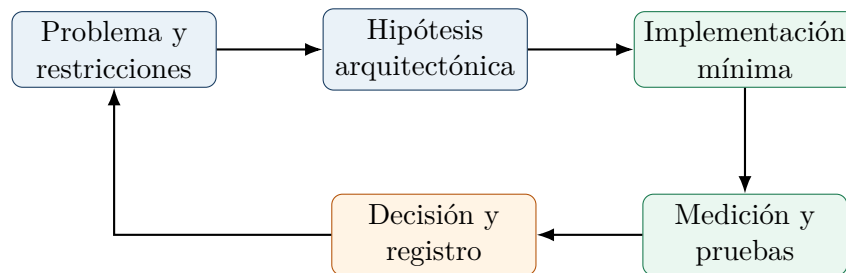


Figura 1.1: Ciclo de decisión arquitectónica guiado por evidencia.

Una hipótesis arquitectónica debe producir una observación. Si se afirma que un índice reducirá el tiempo de consulta, se debe medir el percentil 95 antes y después. Si se afirma que un límite modular permite cambios independientes, se debe demostrar que las pruebas de un módulo pueden ejecutarse sin iniciar todos los demás. La evidencia no elimina el juicio profesional, pero evita decisiones basadas únicamente en intuición.

1.3. Software libre y reproducibilidad

El software libre permite estudiar, modificar y redistribuir el programa bajo las condiciones de su licencia. Esto favorece el aprendizaje porque los componentes pueden ejecutarse localmente e inspeccionarse. Sin embargo, “código disponible” no siempre significa “software libre”, y dos componentes libres pueden imponer obligaciones incompatibles al distribuir una solución. En cada ejercicio se debe registrar nombre, versión, fuente y licencia.

La reproducibilidad requiere que otra persona pueda reconstruir el entorno con instrucciones explícitas. Como mínimo, un repositorio debe contener:

- archivo `README.md` con propósito, requisitos y comandos;
- dependencias declaradas y, cuando sea posible, bloqueadas;
- configuración de ejemplo sin secretos;
- pruebas automáticas y criterios de aceptación;
- licencia del proyecto y atribuciones necesarias;
- registro de decisiones y de uso de IA.

1.4. Preparación del laboratorio

Instalación requerida

Herramientas obligatorias: Git, Python 3.12 o superior, un editor libre (VSCodium, Neovim o Emacs), GNU Make y Podman. Se recomienda un equipo con 8 GB de RAM; los capítulos de observabilidad e IA funcionan mejor con 16 GB.

En Debian o Ubuntu:

```
1 sudo apt-get update
2 sudo apt-get install -y git python3 python3-venv make podman
```

En macOS, Podman recomienda su instalador oficial. Después de instalarlo:

```
1 podman machine init --cpus 4 --memory 6144
2 podman machine start
```

En Windows, instale WSL2 y Podman Desktop. Los comandos del libro deben ejecutarse dentro de la distribución GNU/Linux de WSL2. Documentación: <https://podman.io/docs/installation>.

1.4.1. Repositorio inicial

```
1 mkdir librerereserva && cd librerereserva
2 git init
3 python3 -m venv .venv
4 source .venv/bin/activate
5 printf "# LibreReserva\n" > README.md
6 mkdir -p docs/adr src tests
7 printf ".venv/\n__pycache__/\n.env\n" > .gitignore
8 git add README.md .gitignore docs src tests
9 git commit -m "chore: iniciar proyecto reproducible"
```

Validación

Ejecute:

```
1 git status --short
2 python --version
3 podman info --format '{{.Host.OCIRuntime.Name}}'
4 podman run --rm docker.io/library/hello-world
```

El repositorio debe quedar sin cambios pendientes; Python debe responder con una versión compatible; Podman debe informar un runtime OCI y el contenedor de prueba debe finalizar con código cero.

1.5. Ejercicios

Ejercicio 1. Inventario verificable

Construya una tabla con sistema operativo, arquitectura de CPU, memoria disponible, versiones de Git, Python y Podman, licencia y URL oficial de cada herramienta. Incluya los comandos usados para obtener cada dato. No copie capturas como única evidencia: conserve también la salida textual.

Ejercicio 2. Primera hipótesis

Escriba una hipótesis arquitectónica sobre LibreReserva con la forma: “Si adoptamos X bajo la condición Y, esperamos mejorar Z, medido mediante M, aceptando el costo C”. Identifique qué prueba podría refutarla.

1.6. Lista de comprobación

- El entorno se reconstruye a partir del README.
- Ningún secreto se encuentra dentro del repositorio.
- Las versiones y licencias están registradas.
- Existe al menos una hipótesis que pueda validarse o refutarse.

Capítulo 2

Requisitos, restricciones y atributos de calidad

Objetivos del capítulo

El lector aprenderá a convertir expectativas ambiguas en escenarios verificables, diferenciar requisitos funcionales de atributos de calidad, priorizar tensiones y diseñar funciones de aptitud que revelen degradaciones arquitectónicas.

2.1. Del deseo a la condición comprobable

Los requisitos funcionales describen capacidades: crear una reserva, cancelar un recurso o consultar disponibilidad. Los atributos de calidad describen cómo debe comportarse el sistema: responder con cierta latencia, resistir fallos, impedir accesos indebidos o ser modificable por un equipo pequeño.

Palabras como “rápido”, “seguro” y “escalable” son insuficientes porque no establecen condiciones ni umbrales. Un escenario de atributo de calidad contiene seis partes: fuente del estímulo, estímulo, ambiente, elemento afectado, respuesta y medida.

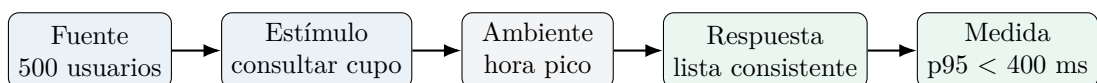


Figura 2.1: Escenario medible de desempeño para LibreReserva.

2.2. Atributos relevantes

2.2.1. Desempeño y escalabilidad

El desempeño se expresa mediante latencia, rendimiento y utilización de recursos. Los promedios ocultan colas largas; por eso suelen observarse percentiles como p95 y p99. Escalabilidad no significa únicamente “agregar servidores”: describe cómo cambia la capacidad cuando varía la carga o los recursos.

2.2.2. Disponibilidad y resiliencia

Disponibilidad es la proporción de tiempo en que una capacidad cumple su objetivo. Resiliencia es la capacidad de absorber, recuperarse y aprender de una perturbación. Reiniciar automáticamente un proceso puede mejorar recuperación, pero no corrige corrupción de datos ni dependencias saturadas.

2.2.3. Seguridad y privacidad

Seguridad incluye confidencialidad, integridad, disponibilidad, autenticidad y trazabilidad. Privacidad considera finalidad, minimización, retención y derechos sobre datos personales. “Cifrar todo” no sustituye control de acceso, gestión de claves ni diseño de flujos.

2.2.4. Modificabilidad y operabilidad

Modificabilidad mide el costo y riesgo de cambiar el sistema. Operabilidad abarca despliegue, diagnóstico, configuración, recuperación y soporte. Un sistema funcional pero imposible de diagnosticar tiene una arquitectura incompleta.

2.3. Restricciones y decisiones

Una restricción limita el espacio de solución: presupuesto, regulación, conocimientos del equipo, infraestructura institucional o fecha. No debe confundirse con una preferencia. “El equipo conoce PostgreSQL” es contexto; “la organización exige PostgreSQL por operación centralizada” es restricción si está formalmente establecida.

Las decisiones generan compromisos. La [table 2.1](#) muestra un análisis cualitativo que debe sustentarse con evidencia.

Cuadro 2.1: Ejemplo de comparación de alternativas.

Criterio	Monolito modular	Microservicios	Serverless
Complejidad inicial	Baja a media	Alta	Media
Consistencia transaccional	Directa	Distribuida	Dependiente de servicios
Despliegue independiente	Por aplicación	Por servicio	Por función
Observabilidad requerida	Moderada	Alta	Alta
Adecuación al equipo pequeño	Alta	Baja sin plataforma	Media

2.4. Funciones de aptitud

Una función de aptitud verifica de manera continua una propiedad arquitectónica. Puede ser una prueba que prohíba dependencias entre módulos, un presupuesto de latencia, una regla de seguridad o una validación de contrato. Debe tener dueño, frecuencia, umbral y respuesta ante incumplimiento.

Ejemplos para LibreReserva:

- ninguna ruta pública puede modificar reservas sin identidad autenticada;
- el módulo de notificaciones no puede importar clases internas de reservas;
- el p95 de consulta debe permanecer por debajo de 400 ms con 100 usuarios concurrentes;
- cada imagen desplegable debe poseer SBOM y no contener vulnerabilidades críticas conocidas sin aceptación documentada.

Laboratorio. Catálogo de escenarios

Identifique cuatro interesados de LibreReserva y redacte ocho escenarios: dos de desempeño, dos de seguridad, uno de disponibilidad, uno de privacidad, uno de modificabilidad y uno de operabilidad. Para cada escenario defina prioridad, riesgo, método de medición y evidencia esperada.

Validación

Un escenario se acepta si otra persona puede reproducir la condición y decidir, sin interpretación adicional, si el umbral se cumplió. Revise que ninguna medida use solamente términos como “rápido”, “adecuado”, “suficiente” o “seguro”.

Ejercicio de decisión

Suponga que LibreReserva debe ser operada por dos personas, servir a una institución y desplegarse en infraestructura propia. Compare monolito modular, microservicios y serverless mediante una matriz ponderada. Realice un análisis de sensibilidad cambiando el peso de disponibilidad y costo operativo.

Riesgo o error frecuente

Las matrices no producen decisiones objetivas: hacen explícitos valores y supuestos. Un puntaje alto no compensa una restricción obligatoria ni una incertidumbre no investigada.

Capítulo 3

Comunicación de arquitectura: C4, UML y ADR

Objetivos del capítulo

El lector podrá seleccionar una vista según su audiencia, producir diagramas C4 coherentes, complementar con UML cuando exista una pregunta dinámica o estructural concreta y registrar decisiones mediante ADR versionados.

3.1. Una vista responde una pregunta

Un diagrama útil declara alcance, audiencia, elementos, relaciones y leyenda. Mezclar procesos de negocio, servidores, clases y proveedores en una sola imagen genera ambigüedad. C4 organiza la estructura estática en niveles de acercamiento: contexto, contenedores, componentes y código. Los dos primeros suelen ser suficientes para discutir arquitectura de solución.

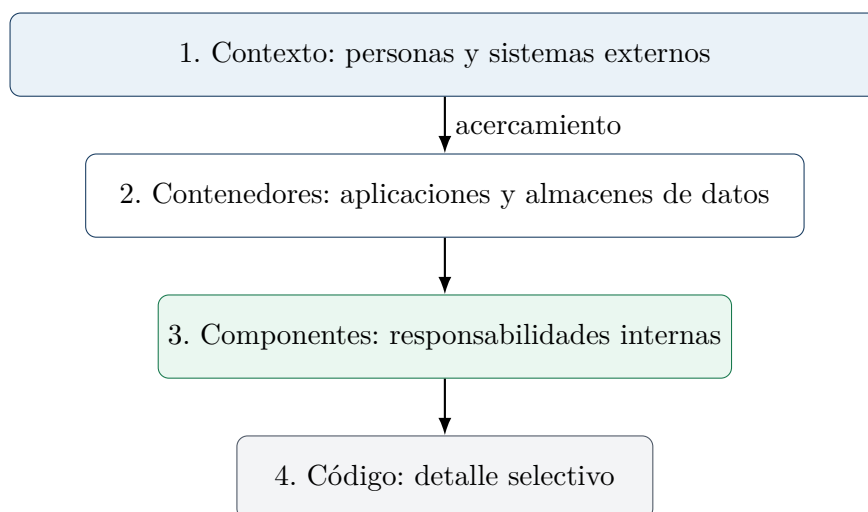


Figura 3.1: Niveles de abstracción del modelo C4.

3.2. Contexto y contenedores de LibreReserva

En contexto aparecen Estudiante, Docente, Administrador, el sistema institucional de identidad y el servicio de correo. No se muestran frameworks ni tablas. La pregunta es quién usa el sistema y con qué sistemas se relaciona.

En contenedores se distinguen interfaz web, API, base de datos y trabajador de notificaciones. Aquí sí se indican responsabilidades, tecnologías y protocolos. “Contenedor” en C4 no significa necesariamente contenedor OCI; puede ser una aplicación, proceso o almacén desplegable.

3.3. Cuándo emplear UML

UML sigue siendo valioso cuando se selecciona el diagrama adecuado:

- secuencia para una interacción crítica, como confirmar una reserva;
- estados para el ciclo solicitada–confirmada–cancelada–expirada;
- clases para un modelo de dominio complejo, sin convertir cada atributo en documentación permanente;
- despliegue para relacionar nodos, procesos y redes cuando C4 no ofrece el detalle requerido.

La consistencia importa más que la notación. Una API llamada *Reservas* en el diagrama de contenedores no debería aparecer como *Booking Core* en una secuencia sin explicar el cambio.

3.4. ADR: decisiones que sobreviven a la reunión

Un Architecture Decision Record registra una decisión significativa. Un formato mínimo contiene título, estado, contexto, opciones, decisión, consecuencias y fecha. Los ADR deben vivir junto al código para que cambien mediante revisión.

```

1  # ADR-0001: iniciar como monolito modular
2
3  Estado: aceptado
4
5  Contexto:
6  Un equipo de cuatro personas debe entregar la primera version en doce semanas.
7
8  Decision:
9  Usar un unico despliegue con modulos Reservas, Recursos, Identidad y Notificaciones.
10
11 Consecuencias:
12 + transacciones y despliegue simples;
13 + menor costo operativo;
14 - se requieren pruebas de limites para evitar acoplamiento interno;
15 - la extraccion futura de un modulo exigira contratos explicitos.
```

3.5. Diagramas como código

Instalación requerida

Instale PlantUML y Graphviz.

Debian/Ubuntu:

```
1 sudo apt-get install -y default-jre graphviz plantuml
```

macOS:

```
1 brew install graphviz plantuml
```

Compruebe con `plantuml -version`. También puede usar Mermaid CLI o Structurizr Lite. No instale extensiones de editor sin revisar permisos y origen.

```
1 @startuml
2 actor Estudiante
3 rectangle "LibreReserva" {
4     [Interfaz web] --> [API de reservas] : HTTPS/JSON
5     [API de reservas] --> database "PostgreSQL" : SQL
6     [API de reservas] --> [Notificaciones] : evento
7 }
8 Estudiante --> [Interfaz web] : consulta y reserva
9 @enduml
```

Guarde el texto como `docs/contexto.puml` y genere la imagen:

```
1 plantuml -tsvg docs/contexto.puml
```

Validación

El SVG debe generarse sin errores. Revise automáticamente que la documentación exista y que corresponda a la fuente versionada:

```
1 test -s docs/contexto.svg
2 git diff --exit-code -- docs/contexto.svg
```

3.6. Ejercicios

Ejercicio 1. Paquete mínimo de arquitectura

Construya para LibreReserva un diagrama de contexto, uno de contenedores, una secuencia de confirmación y un diagrama de estados de Reserva. Cada relación debe estar etiquetada y cada tecnología debe aparecer solamente en el nivel adecuado.

Ejercicio 2. Revisión cruzada

Intercambie los diagramas con otro equipo. Sin explicación oral, identifique usuarios, responsabilidades, protocolos, almacenes, fronteras de confianza y recorrido de una reserva. Registre todas las preguntas que el diagrama no permita responder y modifíquelo sólo cuando la pregunta pertenezca a su alcance.

Ejercicio 3. ADR alternativo

Redacte un ADR que rechace microservicios en la primera versión. Incluya condiciones observables que justificarían reconsiderar la decisión, por ejemplo equipos independientes, ciclos de despliegue incompatibles o requisitos de aislamiento.

Capítulo 4

Modularidad, dominio y patrones

Objetivos del capítulo

El lector podrá diseñar límites modulares, reconocer cohesión y acoplamiento, emplear conceptos de diseño orientado al dominio y construir una arquitectura hexagonal que mantenga el núcleo independiente de interfaces y persistencia.

4.1. Modularidad antes que distribución

Un módulo agrupa responsabilidades que cambian por razones relacionadas y expone un contrato deliberado. La modularidad es una propiedad lógica; no exige múltiples procesos. Un monolito puede estar bien modularizado y un conjunto de servicios puede formar un “monolito distribuido” si todos comparten base de datos, despliegue y conocimiento interno.

La cohesión es la relación interna entre los elementos de un módulo. El acoplamiento es la dependencia entre módulos. El objetivo no es eliminar dependencias, sino hacerlas necesarias, explícitas y estables. Una dependencia hacia una abstracción de negocio suele ser menos volátil que una dependencia hacia una biblioteca concreta de infraestructura.

Para LibreReserva se proponen cuatro contextos iniciales:

- **Recursos:** salas, equipos, capacidades y restricciones;
- **Reservas:** disponibilidad, reglas de solapamiento y ciclo de vida;
- **Identidad:** usuarios, roles y relación con el proveedor institucional;
- **Notificaciones:** preferencias, plantillas y entrega asíncrona.

4.2. Lenguaje ubicuo y límites de dominio

El diseño orientado al dominio propone que desarrolladores y expertos compartan vocabulario. “Reserva confirmada” debe poseer el mismo significado en conversaciones, código, eventos y pruebas. Cuando una palabra cambia de sentido entre áreas, puede indicar una frontera. Por ejemplo, un *recurso* para inventario puede incluir costo de reposición, mientras para reservas sólo interesa capacidad y disponibilidad.

Un agregado protege invariantes dentro de una frontera transaccional. No debe crecer para contener todo lo relacionado. En LibreReserva, la regla “un recurso no puede tener dos reservas

confirmadas que se solapan” requiere una estrategia de concurrencia en Reservas; no obliga a cargar información completa del usuario ni el historial de notificaciones.

4.3. Arquitectura hexagonal

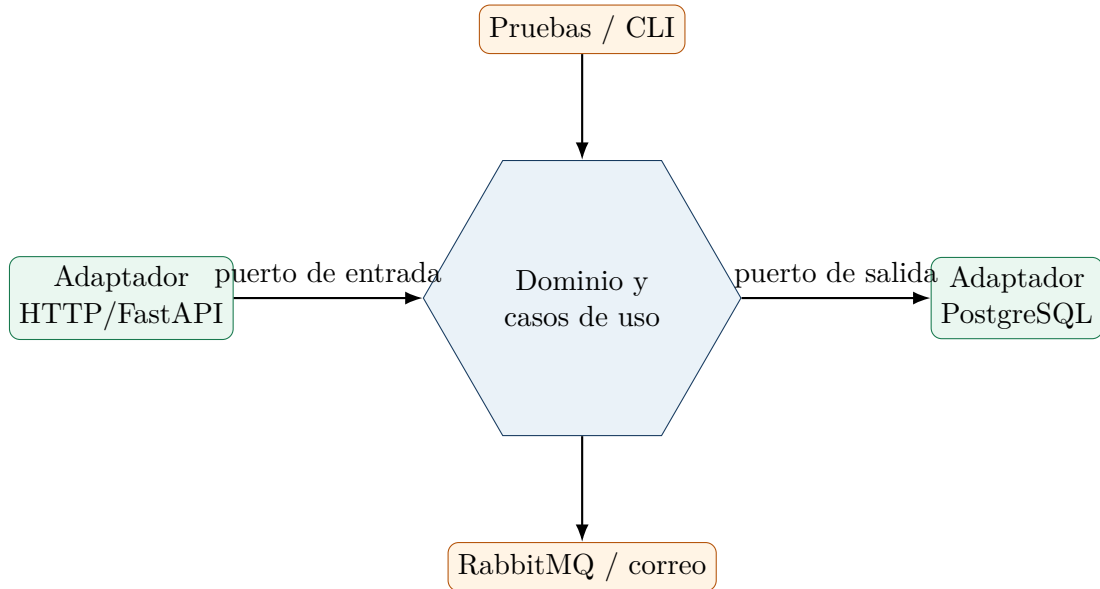


Figura 4.1: El dominio depende de puertos; los adaptadores dependen del dominio.

Los puertos expresan capacidades requeridas o ofrecidas. Un caso de uso puede depender de `RepositorioReservas` sin conocer `SQLAlchemy`. El adaptador `PostgreSQL` implementa ese puerto. Esta dirección permite probar el núcleo con dobles simples y reemplazar infraestructura sin modificar reglas de negocio.

4.4. Patrones y límites

4.4.1. MVC, Repository y DAO

MVC separa entrada, coordinación y presentación, pero no define toda la arquitectura. Repository representa una colección de objetos del dominio; DAO encapsula acceso a datos con una orientación más cercana a tablas o consultas. Usarlos como sinónimos suele producir capas sin significado.

4.4.2. Inversión de dependencias

Las políticas de alto nivel no deberían depender de detalles de bajo nivel. La inversión se concreta cuando el dominio define el contrato y el adaptador lo implementa. Crear una interfaz para cada clase no mejora automáticamente el diseño: la abstracción debe representar variabilidad o un límite real.

4.5. Esqueleto modular en Python

Instalación requerida

Active el entorno virtual creado en el capítulo 1 e instale herramientas de desarrollo:

```
1 python -m pip install --upgrade pip
2 python -m pip install "fastapi[standard-no-fastapi-cloud-cli]" pytest
```

FastAPI y pytest son proyectos de código abierto. Registre versiones con `python -m pip freeze >requirements.lock`.

```
1 # src/reservas/dominio.py
2 from dataclasses import dataclass
3 from datetime import datetime
4
5 @dataclass(frozen=True)
6 class Intervalo:
7     inicio: datetime
8     fin: datetime
9
10     def __post_init__(self):
11         if self.inicio >= self.fin:
12             raise ValueError("el intervalo debe ser positivo")
13
14     def se_solapa(self, otro: "Intervalo") -> bool:
15         return self.inicio < otro.fin and otro.inicio < self.fin
```

```
1 # tests/test_intervalo.py
2 from datetime import datetime
3 import pytest
4 from src.reservas.dominio import Intervalo
5
6 def test_rechaza_intervalo_vacio():
7     instante = datetime(2026, 8, 6, 10, 0)
8     with pytest.raises(ValueError):
9         Intervalo(instante, instante)
10
11 def test_detecta_solapamiento():
12     a = Intervalo(datetime(2026, 8, 6, 9), datetime(2026, 8, 6, 11))
13     b = Intervalo(datetime(2026, 8, 6, 10), datetime(2026, 8, 6, 12))
14     assert a.se_solapa(b)
```

Validación

Ejecute `pytest -q`. Las pruebas del dominio deben funcionar sin iniciar FastAPI, PostgreSQL o RabbitMQ. Esa independencia es evidencia del límite, no sólo una preferencia de carpetas.

4.6. Ejercicios

Ejercicio 1. Mapa de contextos

Defina propietarios, lenguaje, datos y contratos para Recursos, Reservas, Identidad y Notificaciones. Señale relaciones cliente-proveedor y términos que cambian de significado entre contextos.

Ejercicio 2. Prueba de frontera

Cree una prueba que falle si el paquete `src/reservas/dominio.py` importa FastAPI, SQLAlchemy o un cliente de RabbitMQ. Puede analizar el árbol sintáctico con el módulo estándar `ast`. Explique qué tipos de acoplamiento esta prueba no puede detectar.

Ejercicio 3. Concurrencia

Proponga tres estrategias para impedir reservas simultáneas: bloqueo pesimista, control optimista y restricción en base de datos. Compare consistencia, rendimiento, experiencia del usuario y complejidad de recuperación.

Capítulo 5

Estilos arquitectónicos y sistemas distribuidos

Objetivos del capítulo

El lector podrá comparar monolito modular, microservicios, arquitectura orientada a eventos y serverless; identificar costos de distribución; y diseñar una evolución incremental basada en señales observables.

5.1. Los estilos son restricciones deliberadas

Un estilo arquitectónico define un conjunto de elementos, relaciones y restricciones. Capas controla la dirección de dependencias; hexagonal separa política de adaptadores; microservicios agrega autonomía de despliegue y datos; eventos desacopla temporalmente productores y consumidores. Combinar estilos es normal, pero cada combinación añade reglas que deben verificarse.

5.2. Monolito modular

Un monolito modular se despliega como una unidad y contiene límites internos explícitos. Sus ventajas son depuración, transacciones y operación simples. Sus riesgos aparecen cuando los límites se ignoran, la base de datos se convierte en interfaz informal y cada cambio exige comprender todo el sistema.

Para un equipo pequeño y un dominio en aprendizaje, suele ser un punto de partida prudente. La modularidad crea opciones: un módulo puede extraerse después si existe evidencia de que necesita escalar, desplegarse o gobernarse de forma independiente.

5.3. Microservicios

Un microservicio posee una responsabilidad coherente, despliegue independiente y control de sus datos. Dividir por entidades produce servicios anémicos y conversaciones excesivas. El límite debe reflejar capacidad de negocio, equipo y ciclo de cambio.

La distribución introduce latencia, fallos parciales, versionamiento de contratos, observabilidad, seguridad entre servicios, consistencia eventual y automatización. Si dos servicios siempre cambian y se despliegan juntos, la separación puede ser accidental.

5.4. Eventos y mensajería

Un evento declara un hecho ocurrido: **ReservaConfirmada**. No debe ser una orden disfrazada ni incluir el modelo interno completo. Los consumidores pueden reaccionar de forma independiente, pero se deben manejar duplicados, orden, reintentos y evolución del esquema.

5.5. Serverless

Serverless delega infraestructura a una plataforma y factura por uso o capacidad. Puede reducir operación para cargas intermitentes, pero introduce límites de ejecución, arranques, observabilidad distribuida y dependencia del proveedor. Las alternativas libres, como OpenFaaS o Knative, también requieren operar una plataforma subyacente; por tanto, no eliminan el costo, lo redistribuyen.

5.6. Árbol de decisión

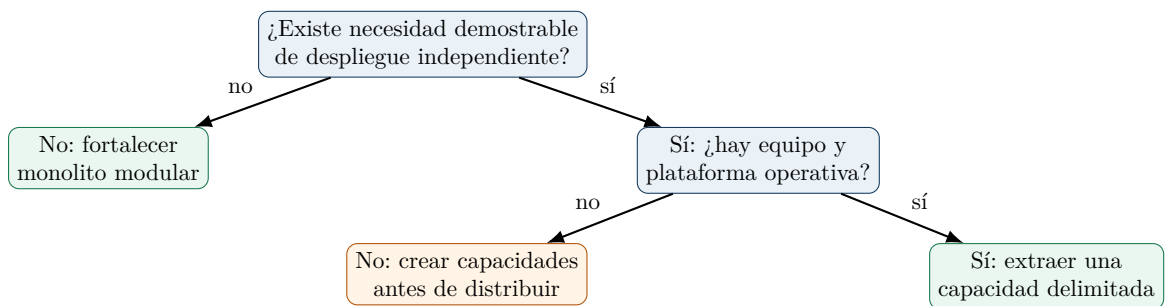


Figura 5.1: Decisión conservadora antes de distribuir.

5.7. Patrones de evolución

- **Strangler**: enrutar gradualmente capacidades hacia un componente nuevo.
- **Anti-corruption layer**: traducir modelos para proteger un dominio de un legado.
- **Outbox**: guardar cambio de negocio y mensaje en una transacción local, publicándolo después.
- **Saga**: coordinar transacciones locales con compensaciones; no restaura automáticamente el estado exacto anterior.
- **Circuit breaker**: detener llamadas a una dependencia fallida para permitir recuperación; no sustituye límites de tiempo.

Laboratorio. Simulación de fallos

Modele la confirmación de una reserva y el envío de correo como dos procesos. Simule que el correo falla en 30 % de los intentos. Compare: transacción síncrona, reintento inmediato, cola con reintento y outbox. Registre reservas confirmadas, correos duplicados y tiempo total.

Validación

La estrategia propuesta debe demostrar: ninguna reserva confirmada se pierde; un fallo de correo no revierte indebidamente la reserva; los mensajes duplicados no producen efectos duplicados; los fallos agotados quedan visibles para operación.

Ejercicio de arquitectura

Seleccione un módulo candidato a extracción. Defina señal de activación, contrato, propiedad de datos, plan de migración, observabilidad y estrategia de reversión. Si no puede identificar una señal medible, argumente por qué debe permanecer en el monolito.

Riesgo o error frecuente

Microservicios no son una medida de madurez. La madurez consiste en poder justificar límites, operar fallos y cambiar con seguridad. Distribuir antes de dominar pruebas, entrega y observabilidad amplifica los problemas existentes.

Capítulo 6

APIs, datos, consistencia y mensajería

Objetivos del capítulo

El lector podrá diseñar contratos HTTP coherentes, emplear OpenAPI, proteger operaciones idempotentes, modelar transacciones en PostgreSQL y usar RabbitMQ sin perder trazabilidad ni consistencia.

6.1. API como contrato

Una API es una promesa entre partes. Su calidad depende de semántica, errores, compatibilidad, seguridad y observabilidad, no sólo de rutas. En HTTP, GET consulta sin cambiar estado observable; POST crea o ejecuta una operación; PUT reemplaza de forma idempotente; PATCH modifica parcialmente; DELETE solicita eliminación.

Los códigos de estado deben ayudar al cliente: 400 para solicitud inválida, 401 para falta de autenticación, 403 para autorización insuficiente, 404 para ausencia, 409 para conflicto y 422 para datos semánticamente inválidos. Devolver siempre 200 y esconder errores en JSON dificulta monitoreo y clientes genéricos.

6.2. Idempotencia y concurrencia

Una operación idempotente produce el mismo efecto al repetirse. Las redes reintentan; por ello, una creación crítica puede aceptar una clave de idempotencia. El servidor conserva la relación entre clave, identidad, operación y respuesta. La clave no debe compartirse entre usuarios ni aceptarse indefinidamente.

Para evitar solapamientos, PostgreSQL permite transacciones, niveles de aislamiento y restricciones. Una comprobación “consultar y luego insertar” sin protección es vulnerable a carreras: dos transacciones pueden ver disponibilidad antes de insertar.

6.3. Persistencia y propiedad de datos

La base de datos es parte de la arquitectura. Cada módulo debe definir qué tablas controla y qué consultas expone. Compartir tablas permite velocidad inicial, pero crea contratos invisibles. Las

migraciones deben ser compatibles con la versión desplegada, reversibles cuando sea posible y probadas con datos representativos.

6.4. Mensajería asíncrona

RabbitMQ implementa colas e intercambios. Un productor publica; el broker enruta; un consumidor procesa y confirma. “Exactamente una vez” rara vez es una garantía extremo a extremo. El diseño práctico asume entrega al menos una vez y hace idempotente al consumidor.

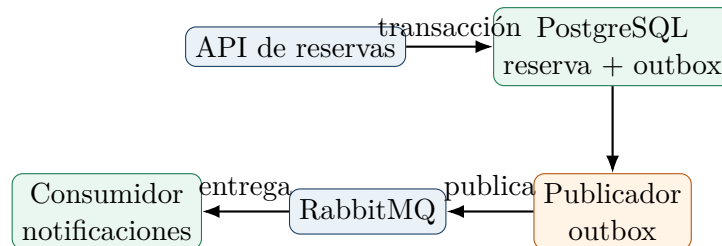


Figura 6.1: Patrón outbox para publicación confiable.

6.5. Plataforma local

Instalación requerida

Se requiere Podman. Cree una red y ejecute PostgreSQL y RabbitMQ con credenciales exclusivas de laboratorio:

```

1 podman network create librerereserva
2 podman run -d --name postgres --network librerereserva \
3   -e POSTGRES_DB=reservas -e POSTGRES_USER=reservas \
4   -e POSTGRES_PASSWORD=solo-laboratorio \
5   -p 5432:5432 docker.io/library/postgres:17
6 podman run -d --name rabbit --network librerereserva \
7   -p 5672:5672 -p 15672:15672 \
8   docker.io/library/rabbitmq:4-management
  
```

No reutilice estas credenciales en otros entornos. Para producción use secretos externos, TLS, usuarios con privilegios mínimos e imágenes fijadas por digest.

```

1 from fastapi import FastAPI, Header, HTTPException, status
2 from pydantic import BaseModel
3
4 app = FastAPI(title="LibreReserva", version="1.0.0")
5 respuestas: dict[str, dict] = {}
6
7 class NuevaReserva(BaseModel):
8     recurso_id: str
9     inicio: str
10    fin: str
11
12 @app.post("/reservas", status_code=status.HTTP_201_CREATED)
13 def crear(datos: NuevaReserva, idempotency_key: str = Header()):
14     if idempotency_key in respuestas:
15         return respuestas[idempotency_key]
  
```

```
16     if datos.inicio >= datos.fin:
17         raise HTTPException(422, "intervalo invalido")
18     respuesta = {"id": idempotency_key, "estado": "confirmada"}
19     respuestas[idempotency_key] = respuesta
20     return respuesta
```

Validación

Inicie la API con `fastapi dev`. Consulte <http://127.0.0.1:8000/docs>. Envíe dos POST con la misma clave; deben devolver el mismo identificador y no crear dos efectos. Detenga RabbitMQ y compruebe que la reserva pueda persistirse con un registro outbox pendiente.

6.6. Ejercicios

Ejercicio 1. Contrato OpenAPI

Defina rutas para consultar recursos, crear, cancelar y consultar reservas. Incluya esquemas de error, autenticación, paginación y claves de idempotencia. Valide el documento con una herramienta OpenAPI libre y genere pruebas de ejemplos inválidos.

Ejercicio 2. Consumidor idempotente

Implemente una tabla de mensajes procesados. Entregue dos veces `ReservaConfirmada`; el consumidor debe enviar una sola notificación. Explique qué ocurre si el proceso cae después de enviar el correo y antes de confirmar el mensaje.

Ejercicio 3. Compatibilidad

Agregue un campo opcional `proposito` al evento. Demuestre compatibilidad entre productor nuevo y consumidor antiguo. Después conviértalo en obligatorio y documente por qué el cambio rompe consumidores.

Capítulo 7

Implementación evolutiva y pruebas automatizadas

Objetivos del capítulo

Al finalizar este capítulo, el estudiante podrá construir un incremento vertical pequeño, separar pruebas unitarias, de integración y de aceptación, medir cobertura sin confundirla con calidad y definir puertas automáticas de validación.

7.1. Construir por incrementos verificables

Una arquitectura solo existe de manera útil cuando sus decisiones pueden verse en código ejecutable. Un *incremento vertical* atraviesa interfaz, lógica de aplicación y persistencia para entregar una capacidad observable. Es preferible implementar una reserva completa y pequeña que crear, por separado, muchas capas aún inconexas.

El ciclo recomendado es: formular una hipótesis, escribir una prueba que represente el comportamiento, implementar lo mínimo, refactorizar y registrar la evidencia. La prueba no sustituye el razonamiento de diseño; lo vuelve repetible.

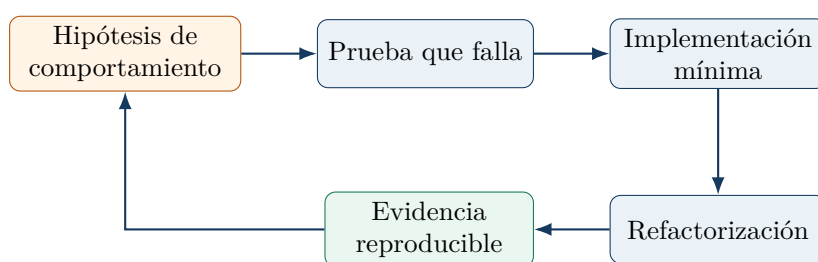


Figura 7.1: Ciclo corto de implementación y validación.

7.2. Una estrategia de pruebas por riesgo

La pirámide de pruebas es una guía económica, no una cuota fija. Las pruebas unitarias aíslan reglas rápidas; las de integración verifican contratos con base de datos, cola o sistema de archivos; las de extremo a extremo recorren el sistema desplegado. Cuanto más amplia sea la prueba, mayor suele ser su costo de preparación, tiempo y diagnóstico.

Cuadro 7.1: Selección de pruebas según el riesgo.

Nivel	Pregunta principal	Ejemplo en LibreReserva
Unitaria	¿La regla produce el resultado correcto?	Rechazar un intervalo que termina antes de comenzar.
Integración	¿La colaboración real cumple el contrato?	Guardar una reserva y detectar solapamiento en PostgreSQL.
Contrato	¿Consumidor y proveedor interpretan igual la interfaz?	Verificar esquema y códigos de la API.
Aceptación	¿El flujo entrega valor desde fuera?	Crear, consultar y cancelar una reserva por HTTP.
Rendimiento	¿Se mantiene el objetivo bajo carga?	Percentil 95 menor al umbral con concurrencia definida.
Seguridad	¿Los controles resisten usos adversos?	Negar acceso a reservas de otro usuario.

7.3. Dobles, propiedades y datos de prueba

Un doble de prueba reemplaza una dependencia para controlar el entorno. Un *stub* responde valores preparados, un *fake* ofrece una implementación simplificada y un *mock* verifica interacciones. Deben usarse en los bordes; simular cada objeto interno acopla las pruebas a la implementación.

Las pruebas basadas en propiedades expresan invariantes para muchos datos generados. Por ejemplo: toda reserva aceptada tiene duración positiva y ninguna secuencia de creación puede dejar dos reservas activas solapadas para el mismo recurso. En Python, Hypothesis es una opción libre para generar casos y reducir automáticamente un fallo al ejemplo mínimo.

Instalación requerida

Se requiere Python 3.11 o superior. En un entorno virtual instale las herramientas libres:

```

1 python3 -m venv .venv
2 source .venv/bin/activate           # Windows PowerShell: .venv\Scripts\
   Activate.ps1
3 python -m pip install fastapi uvicorn pytest pytest-cov httpx hypothesis

```

No use `sudo pip`. El entorno virtual hace reproducible el laboratorio y evita modificar el Python del sistema.

7.4. Ejemplo: contrato HTTP comprobable

Código 7.1: API mínima con validación explícita.

```

1 from datetime import datetime
2 from fastapi import FastAPI, HTTPException, status
3 from pydantic import BaseModel, model_validator
4
5 app = FastAPI(title="LibreReserva")
6 reservas: list[dict] = []
7
8 class ReservaEntrada(BaseModel):
9     recurso: str

```

```

10     inicio: datetime
11     fin: datetime
12
13     @model_validator(mode="after")
14     def intervalo_valido(self):
15         if self.fin <= self.inicio:
16             raise ValueError("fin debe ser posterior a inicio")
17         return self
18
19 @app.post("/reservas", status_code=status.HTTP_201_CREATED)
20 def crear(entrada: ReservaEntrada):
21     hay_cruce = any(
22         r["recurso"] == entrada.recurso
23         and entrada.inicio < r["fin"] and r["inicio"] < entrada.fin
24         for r in reservas
25     )
26     if hay_cruce:
27         raise HTTPException(status_code=409, detail="intervalo ocupado")
28     nueva = {"id": len(reservas) + 1, **entrada.model_dump()}
29     reservas.append(nueva)
30     return nueva

```

Código 7.2: Pruebas externas del comportamiento HTTP.

```

1 from fastapi.testclient import TestClient
2 from app import app, reservas
3
4 cliente = TestClient(app)
5
6 def setup_function():
7     reservas.clear()
8
9 def test_crea_y_rechaza_solapamiento():
10     datos = {"recurso": "A-101",
11             "inicio": "2026-08-10T08:00:00Z",
12             "fin": "2026-08-10T09:00:00Z"}
13     assert cliente.post("/reservas", json=datos).status_code == 201
14     assert cliente.post("/reservas", json=datos).status_code == 409
15
16 def test_rechaza_intervalo_invertido():
17     datos = {"recurso": "A-101",
18             "inicio": "2026-08-10T10:00:00Z",
19             "fin": "2026-08-10T09:00:00Z"}
20     assert cliente.post("/reservas", json=datos).status_code == 422

```

7.5. Cobertura, mutación y calidad

La cobertura indica qué código fue ejecutado, no si fue correctamente comprobado. Una cobertura alta puede coexistir con aserciones débiles. La prueba de mutación cambia deliberadamente operadores o constantes: si la suite no detecta el cambio, falta sensibilidad. Para un curso corto es suficiente combinar cobertura de ramas, revisión de aserciones y una mutación manual de una regla crítica.

Validación

Ejecute `pytest -cov=. -cov-branch -cov-report=term-missing`. Deben aprobarse todas las pruebas y cubrirse ambas decisiones del solapamiento. Cambie temporalmente `entrada.inicio <r["fin"]` por `<=`; documente qué prueba falla. Restaure el código antes de confirmar cambios.

Ejercicio propuesto

Laboratorio 7.1 — incremento vertical. En Git y FastAPI implemente consulta y cancelación. Añada pruebas unitarias de dominio, integración HTTP y al menos una propiedad con Hypothesis. Entregue código, informe de cobertura y una nota sobre un defecto descubierto por las pruebas.

Laboratorio 7.2 — presupuesto de rendimiento. Instale Locust con `python -m pip install locust`. Modele una carga reproducible, mida tasa de solicitudes, errores y p95, y grafique el resultado. Declare máquina, versión, datos y duración: sin esas condiciones la cifra no es comparable.

7.6. Preguntas de revisión

1. ¿Qué riesgo quedaría invisible si solo se escribieran pruebas unitarias?
2. ¿Por qué un porcentaje de cobertura no debe convertirse por sí solo en objetivo de aprendizaje?
3. ¿Qué evidencia distingue una prueba inestable de un defecto intermitente real?

Capítulo 8

Entrega continua, contenedores e infraestructura reproducible

Objetivos del capítulo

El estudiante podrá convertir una revisión de código en un artefacto trazable, construir imágenes sin privilegios con Podman, expresar una canalización en Forgejo Actions y desplegar un servicio en un clúster local creado con kind.

8.1. Del cambio al servicio operable

La entrega continua mantiene el sistema en estado desplegable. Cada cambio pequeño atraviesa las mismas verificaciones, produce un artefacto inmutable y deja una relación entre confirmación, imagen, configuración y despliegue. Automatizar no significa desplegar sin criterio: producción puede conservar una aprobación humana basada en evidencia.

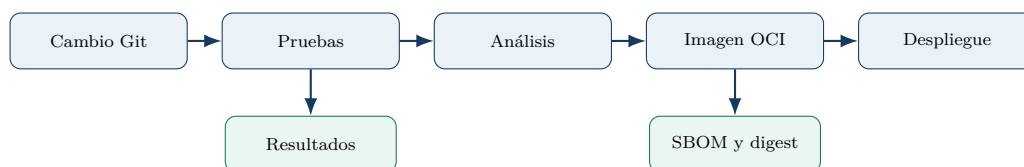


Figura 8.1: Cadena mínima de evidencia de entrega.

8.2. Git como protocolo de colaboración

Una rama corta reduce divergencia. La solicitud de cambio explica intención, riesgo y validación; la revisión examina comportamiento y decisiones, no solo estilo. Una confirmación debe representar una unidad coherente. Etiquetas y versiones semánticas comunican compatibilidad, pero no reemplazan notas de cambio.

Validación

Antes de integrar: el árbol está limpio; las pruebas son verdes; no hay secretos; la documentación del contrato cambió cuando correspondía; la decisión arquitectónica significativa tiene ADR; otra persona puede reproducir el resultado desde el repositorio.

8.3. Contenedores sin demonio privilegiado

Una imagen OCI empaqueta sistema de archivos y metadatos; un contenedor es un proceso aislado iniciado desde ella. La imagen debe ser pequeña, ejecutarse con usuario no privilegiado y recibir configuración en tiempo de ejecución. No deben hornearse secretos ni archivos de desarrollo.

Instalación requerida

Instale Podman desde los repositorios de su distribución o desde <https://podman.io/docs/installation>. En macOS y Windows, Podman usa una máquina Linux:

```
1 podman machine init      # solo la primera vez
2 podman machine start
3 podman info
```

Para alojamiento Git libre, Forgejo publica imágenes de contenedor e instrucciones en <https://forgejo.org/docs/v16.0/admin/installation/docker/>. El laboratorio puede utilizar una instancia institucional existente o una local; no exige GitHub.

Código 8.1: Containerfile reproducible para la API.

```
1 FROM docker.io/library/python:3.13-slim
2 ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
3 WORKDIR /app
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir --require-hashes -r requirements.txt
6 COPY app ./app
7 USER 10001:10001
8 EXPOSE 8000
9 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

La opción `-require-hashes` requiere fijar versión y hash de cada dependencia. Si el equipo aún no mantiene un archivo bloqueado, debe generarlo conscientemente antes de activar esta puerta; retirar la opción sin documentarlo debilita la reproducibilidad.

```
1 podman build -t localhost/librerereserva:0.1.0 .
2 podman run --rm -p 8000:8000 localhost/librerereserva:0.1.0
3 curl -fsS http://localhost:8000/docs >/dev/null
4 podman image inspect localhost/librerereserva:0.1.0 --format '{{.Digest}}'
```

8.4. Integración continua con Forgejo Actions

Forgejo Actions interpreta flujos declarativos compatibles en buena medida con el formato conocido de GitHub Actions, pero requiere un *runner*. Las acciones de terceros son código: se fijan a una versión o, para mayor control, a una confirmación revisada.

Código 8.2: `.forgejo/workflows/validar.yml`

```
1 name: validar
2 on: [push, pull_request]
3 jobs:
4   pruebas:
5     runs-on: docker
6     steps:
```

```

7   - uses: actions/checkout@v4
8   - uses: actions/setup-python@v5
9     with:
10      python-version: '3.13'
11   - run: python -m pip install -r requirements-dev.txt
12   - run: pytest --cov=app --cov-branch
13   - run: podman build -t localhost/librerereserva:${{ forgejo.sha }} .

```

El ejemplo debe ajustarse a las etiquetas y capacidades del *runner*: no todos exponen Podman. En entornos compartidos conviene separar construcción de imágenes en un ejecutor aislado.

8.5. Despliegue local con Kubernetes

Kubernetes reconcilia estado deseado y observado. No corrige una aplicación sin límites ni buenas señales; añade mecanismos de programación, aislamiento y recuperación. Para aprender el modelo sin depender de una nube, kind crea nodos Kubernetes como contenedores.

Instalación requerida

Instale `kubectl` y `kind` siguiendo <https://kind.sigs.k8s.io/docs/user/quick-start/>. Con Podman, kind puede detectar el proveedor; si hay ambigüedad use `KIND_EXPERIMENTAL_PROVIDER=podman`. Verifique versiones antes de crear el clúster.

Código 8.3: Recursos mínimos de despliegue.

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata: {name: librerereserva}
4  spec:
5    replicas: 2
6    selector: {matchLabels: {app: librerereserva}}
7    template:
8      metadata: {labels: {app: librerereserva}}
9      spec:
10       containers:
11         - name: api
12           image: localhost/librerereserva:0.1.0
13           ports: [{containerPort: 8000}]
14           resources:
15             requests: {cpu: 100m, memory: 128Mi}
16             limits: {cpu: 500m, memory: 256Mi}
17           readinessProbe:
18             httpGet: {path: /salud/lista, port: 8000}
19           livenessProbe:
20             httpGet: {path: /salud/viva, port: 8000}
21 ---
22 apiVersion: v1
23 kind: Service
24 metadata: {name: librerereserva}
25 spec:
26   selector: {app: librerereserva}
27   ports: [{port: 80, targetPort: 8000}]

```

```
1 kind create cluster --name informati1
2 kind load docker-image localhost/librerereserva:0.1.0 --name informati1
3 kubectl apply -f despliegue.yml
4 kubectl rollout status deployment/librerereserva --timeout=120s
5 kubectl get pods,service
```

Aunque el subcomando se llama `docker-image`, `kind` lo conserva por compatibilidad. Si la integración con Podman de su plataforma no carga la imagen, utilice un registro local documentado en la guía de `kind`.

8.6. Configuración, secretos e infraestructura como código

Código, configuración no secreta y secretos tienen ciclos distintos. Los secretos se entregan por un gestor o mecanismo de la plataforma, se rotan y nunca se confirman en Git. La infraestructura como código expresa recursos revisables y repetibles; OpenTofu es una alternativa libre para provisión declarativa y Ansible para configuración. En el curso se evalúa la intención, el plan y la revisión, no el gasto en una nube comercial.

Ejercicio propuesto

Laboratorio 8.1 — canalización local. Aloje LibreReserva en Forgejo, active un runner aislado y configure pruebas, cobertura y construcción OCI. Entregue registro de ejecución, digest de imagen y explicación de una puerta fallida.

Laboratorio 8.2 — despliegue reproducible. Cree un clúster `kind`, despliegue dos réplicas, elimine un pod y observe la reconciliación. Pruebe por separado *readiness* y *liveness*; explique por qué no deben apuntar necesariamente al mismo criterio.

8.7. Preguntas de revisión

1. ¿Qué evidencia permite reconstruir qué código está ejecutándose?
2. ¿Por qué una imagen que funciona como `root` sigue siendo un riesgo?
3. ¿Qué costo introduce Kubernetes que un monolito sencillo quizá no justifique?

Capítulo 9

Observabilidad, confiabilidad y costo

Objetivos del capítulo

El estudiante podrá instrumentar señales correlacionadas, formular indicadores y objetivos de servicio, usar presupuestos de error y validar hipótesis operativas con OpenTelemetry, Prometheus, Grafana y Jaeger.

9.1. Observar para explicar

Monitorear responde preguntas previstas; observar permite investigar estados no anticipados a partir de la telemetría. Los registros describen eventos, las métricas resumen series numéricas y las trazas conectan el recorrido de una solicitud. Ninguna señal basta por sí sola. Un identificador de traza común permite pasar de una alerta agregada a una operación concreta.

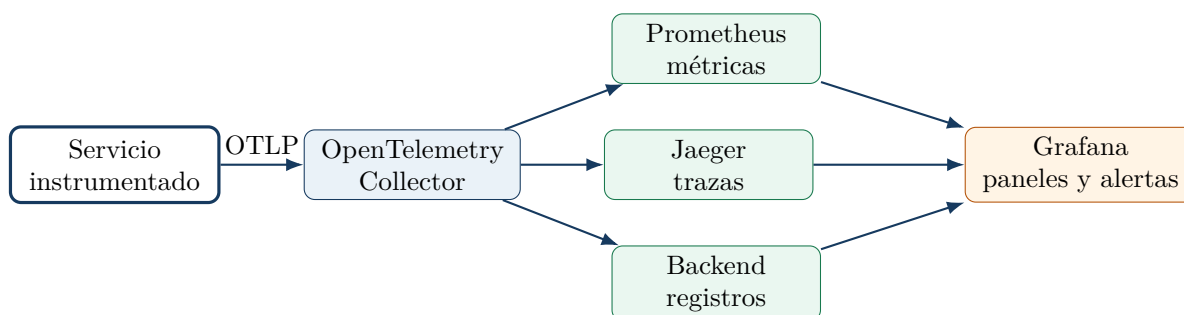


Figura 9.1: Flujo conceptual de telemetría; los backends concretos son sustituibles.

9.2. Diseño de señales

Para servicios orientados a solicitudes, RED propone tasa, errores y duración. Para recursos, USE propone utilización, saturación y errores. Una métrica debe tener nombre, unidad, descripción y cardinalidad controlada. Etiquetar con identificador de usuario o URL completa crea series potencialmente ilimitadas y puede exponer datos personales.

Los registros estructurados facilitan consulta. Deben incluir marca temporal, nivel, evento, componente e identificadores de correlación, pero no tokens, contraseñas, cuerpos completos ni datos personales innecesarios. Las trazas se muestrean: conservar el 100 % puede ser inviable; el muestreo debe mantener suficientes errores y casos lentos para investigar.

9.3. SLI, SLO y presupuesto de error

Un indicador de nivel de servicio (SLI) cuantifica una experiencia: proporción de solicitudes válidas atendidas correctamente antes de 400 ms. Un objetivo (SLO) fija el nivel esperado en una ventana: 99,5 % durante 28 días. El acuerdo (SLA) añade consecuencias contractuales y no debe confundirse con el objetivo interno.

Si el SLO mensual es 99,5 %, el presupuesto de error es 0,5 %. Para una ventana de 28 días, esto equivale a:

$$28 \times 24 \times 60 \times 0,005 = 201,6 \text{ minutos.}$$

El cálculo no autoriza consumir indiscriminadamente esa indisponibilidad; sirve para equilibrar confiabilidad y velocidad de cambio.

Cuadro 9.1: Especificación de un SLI útil.

Elemento	Definición para LibreReserva
Evento válido	Solicitud HTTP a crear o consultar reserva, excluyendo tráfico sintético identificado.
Evento bueno	Respuesta no 5xx completada en menos de 400 ms.
SLI	Eventos buenos divididos por eventos válidos.
SLO	Al menos 99,5 % en ventana móvil de 28 días.
Fuente	Métricas de servidor instrumentadas con OpenTelemetry.
Respuesta	Congelar cambios riesgosos si la tasa de consumo amenaza agotar el presupuesto.

9.4. Instrumentación libre y neutral

OpenTelemetry define APIs, SDK, convenciones y un protocolo para producir y transportar telemetría sin atar el código a un proveedor. El Collector recibe, procesa y exporta señales. Prometheus consulta y almacena métricas; Grafana visualiza; Jaeger permite explorar trazas. La demostración oficial de OpenTelemetry ofrece un sistema distribuido deliberadamente complejo para experimentar.

Instalación requerida

Para la aplicación Python:

```
1 python -m pip install opentelemetry-distro \
2   opentelemetry-exporter-otlp
3 opentelemetry-bootstrap -a install
```

Para un laboratorio integrado use Podman Compose o Docker Compose y los archivos de la demostración oficial: <https://opentelemetry.io/docs/demo/>. Prometheus documenta sus imágenes en <https://prometheus.io/docs/prometheus/latest/installation/>; Grafana, en <https://grafana.com/docs/grafana/latest/setup-grafana/configure-docker/>. Compruebe puertos disponibles y memoria: el demo completo puede exceder 6 GiB; para equipos limitados despliegue solo aplicación, Collector, Prometheus y Jaeger.

```

1 export OTEL_SERVICE_NAME=librerereserva-api
2 export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
3 export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
4 opentelemetry-instrument uvicorn app.main:app --port 8000

```

9.5. Validar con una hipótesis operativa

Un panel decorativo no constituye observabilidad. Cada panel debe apoyar una decisión. Ejemplo: “el aumento del p95 se origina en esperas de base de datos”. La métrica identifica la ventana; una traza lenta muestra el tramo; el registro estructurado confirma el evento; una prueba controlada valida la causa.

Validación

Genere diez solicitudes correctas, una que produzca conflicto 409 y, en una rama experimental, una latencia artificial. Verifique: (1) la métrica de tasa aumenta; (2) 409 no se clasifica automáticamente como fallo del servidor; (3) la traza lenta contiene el tramo responsable; (4) el registro comparte `trace_id`; (5) ningún atributo contiene secretos o datos personales.

9.6. Rendimiento, resiliencia y costo

La latencia se informa con percentiles, no solo promedio. Un límite de tiempo evita esperas infinitas; un reintento solo es seguro cuando la operación lo permite, usa retroceso y dispersión, y tiene un presupuesto. De lo contrario amplifica una sobrecarga. El circuito de corte, el aislamiento de recursos y la degradación controlada reducen propagación de fallos.

El costo también es atributo arquitectónico. Réplicas o trazas adicionales pueden mejorar confiabilidad y elevar consumo. Debe reportarse por unidad útil —por ejemplo, costo estimado por mil reservas— y relacionarse con el SLO. En laboratorio se aceptan aproximaciones basadas en CPU, memoria y almacenamiento; deben explicitar supuestos.

Ejercicio propuesto

Laboratorio 9.1 — pregunta antes que panel. Instrumente LibreReserva, construya un panel RED y documente una consulta de diagnóstico que enlace métrica, traza y registro. Incluya capturas, configuración versionada y explicación; no evalúe estética aislada.

Laboratorio 9.2 — experimento de resiliencia. Inyecte demora y fallos a una dependencia local, compare sin y con timeout/reintento acotado y grafique p50, p95, errores y volumen adicional. Concluya cuándo el reintento ayudó y cuándo empeoró el sistema.

9.7. Preguntas de revisión

1. ¿Qué etiqueta de alta cardinalidad eliminaría y qué alternativa conservaría capacidad diagnóstica?
2. ¿Cómo cambia una decisión de despliegue cuando queda 10% del presupuesto de error?
3. ¿Por qué el promedio de latencia puede ocultar una mala experiencia real?

Capítulo 10

Seguridad, privacidad y cadena de suministro

Objetivos del capítulo

El estudiante podrá modelar amenazas, diseñar controles desde los requisitos, gestionar secretos, explorar vulnerabilidades de aplicación y dependencias, producir una SBOM y justificar la prioridad de remediación con evidencia.

10.1. Seguridad como propiedad del sistema

La seguridad no es una herramienta ejecutada al final. Surge de decisiones sobre identidad, datos, límites de confianza, operación y recuperación. *Secure by Design* exige que los resultados seguros sean la opción normal; *Secure by Default*, que una instalación inicial no dependa de que cada usuario descubra ajustes esenciales.

El marco SSDF de NIST agrupa prácticas para preparar la organización, proteger el software, producir software bien protegido y responder a vulnerabilidades. En el curso se traduce en criterios verificables: revisión, dependencias fijadas, mínimos privilegios, análisis automatizado, procedencia y plan de respuesta.

10.2. Modelado de amenazas

Un modelo de amenazas comienza con activos y límites, no con una lista genérica de ataques. Para LibreReserva son activos las identidades, disponibilidad de espacios, reservas, historial y credenciales. El diagrama de flujo de datos identifica procesos, almacenes, actores y cruces de confianza.

STRIDE ayuda a preguntar por suplantación, manipulación, repudio, divulgación, denegación y elevación de privilegios. No calcula riesgo. Cada amenaza necesita precondition, impacto, control, propietario y prueba. Una escala simple de probabilidad e impacto facilita ordenar el trabajo, sin pretender una precisión inexistente.

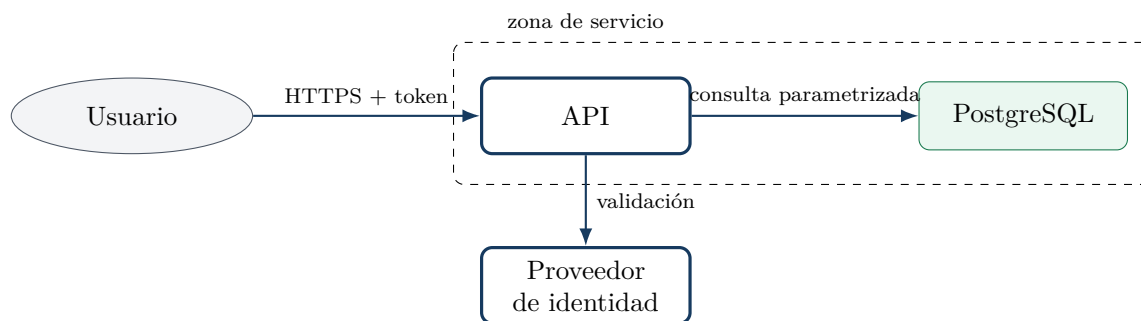


Figura 10.1: Flujo de datos simplificado y límite de confianza.

Cuadro 10.1: Extracto del modelo de amenazas.

Amenaza	Activo	Control diseñado	Validación
Acceso horizontal	Reserva ajena	Autorizar por sujeto y recurso en cada operación	Prueba con dos identidades; esperar 404 o 403 sin filtrar datos.
Inyección	Base de datos	Consultas parametrizadas y cuenta con privilegio mínimo	Casos adversos y análisis dinámico.
Token filtrado	Identidad	No registrar cabeceras sensibles; expiración y rotación	Escaneo de registros y repositorio.
Agotamiento	Disponibilidad	Cuotas, límites, timeout y presión inversa	Carga controlada y alerta antes de saturación.

10.3. Identidad, autorización y secretos

Autenticar confirma una identidad; autorizar decide una acción sobre un recurso. La verificación debe ocurrir en el servidor, incluso si la interfaz oculta botones. Para permisos ricos, una política explícita es más auditable que condicionales dispersos. Las cuentas de servicio reciben mínimo privilegio y una identidad distinta por carga.

Un secreto no es configuración ordinaria. Debe generarse con entropía suficiente, almacenarse fuera del repositorio, limitarse, rotarse y revocarse. Un archivo `.env` local puede facilitar desarrollo, pero se excluye de Git y se acompaña de `.env.example` sin valores reales. Si un secreto fue confirmado, borrarlo del último cambio no basta: se revoca y se considera expuesto en todo el historial distribuido.

10.4. Privacidad y abuso

La minimización reduce superficie: recopilar solo lo necesario, conservarlo el tiempo definido y limitar usos. El equipo registra propósito, base de autorización institucional, acceso, retención y mecanismo de eliminación. Los registros de observabilidad también son datos y deben respetar estas reglas.

Los casos de abuso complementan los casos de uso: acaparar todos los horarios, enumerar reservas, automatizar cancelaciones o inferir ocupación sensible. Los controles pueden combinar reglas de negocio, límites de tasa, detección y revisión, evitando bloquear injustamente usuarios legítimos.

10.5. Dependencias y procedencia

Una lista de materiales de software (SBOM) enumera componentes y versiones. No prueba que sean seguros, pero permite responder si una vulnerabilidad afecta al producto. El análisis de composición, el escaneo de imágenes y la detección de secretos son señales; los hallazgos deben verificarse según explotabilidad, exposición y compensaciones.

Instalación requerida

Las siguientes herramientas tienen código abierto y ofrecen binarios o imágenes oficiales. Consulte las instrucciones de su plataforma antes de ejecutar instaladores:

- Trivy: <https://www.trivy.dev/docs/latest/getting-started/installation/>.
- Syft: <https://github.com/anchore/syft>.
- Gitleaks: <https://github.com/gitleaks/gitleaks>.
- OWASP ZAP en contenedor: <https://www.zaproxy.org/docs/docker/about/>.

En Debian/Ubuntu o macOS use el paquete oficial correspondiente; registre versión y origen. Una imagen puede ejecutarse con Podman sustituyendo `docker` en los ejemplos del proyecto cuando el comando sea compatible.

Código 10.1: Controles reproducibles sobre repositorio e imagen.

```
1 gitleaks detect --source . --redact --report-format sarif \
2   --report-path resultados/gitleaks.sarif
3 syft localhost/librerereserva:0.1.0 -o cyclonedx-json \
4   > resultados/sbom.cdx.json
5 trivy image --severity HIGH,CRITICAL \
6   --format json --output resultados/trivy.json \
7   localhost/librerereserva:0.1.0
8 podman run --rm --network host -v "$PWD/resultados:/zap/wrk:Z" \
9   ghcr.io/zaproxy/zaproxy:stable zap-baseline.py \
10  -t http://127.0.0.1:8000 -J zap.json
```

La red del contenedor cambia entre sistemas; `--network host` no se comporta igual en todas las máquinas virtuales. Si ZAP no alcanza la aplicación, use el nombre o dirección documentada por Podman y registre el ajuste. Nunca apunte un escáner activo a un sistema externo sin autorización expresa.

Validación

La canalización debe fallar ante un secreto de prueba reconocido, pero la demostración se realiza con un patrón ficticio provisto por la herramienta, nunca con credenciales reales. La SBOM debe contener el paquete web esperado. Para cada vulnerabilidad alta: confirmar versión, ruta, exposición y corrección disponible; aceptar temporalmente exige justificación, responsable y fecha de vencimiento.

10.6. Respuesta y recuperación

El diseño incluye detección, contención, erradicación, recuperación y aprendizaje. Se conserva evidencia con acceso controlado, se rota lo comprometido, se informa por canales institucionales y se escribe una revisión sin culpabilizar personas. Copias de seguridad solo cuentan si una

restauración ha sido ensayada.

Ejercicio propuesto

Laboratorio 10.1 — amenaza a control. Modele el flujo de LibreReserva, identifique ocho amenazas STRIDE, priorice cuatro y escriba una prueba por control. Entregue diagrama, registro de amenazas y resultados.

Laboratorio 10.2 — cadena de suministro. Genere SBOM CycloneDX, escanee imagen y repositorio, ejecute ZAP Baseline contra la instancia local y triage cinco hallazgos. Distinga vulnerabilidad confirmada, no aplicable, riesgo aceptado y falso positivo; no entregue solo reportes automáticos.

10.7. Preguntas de revisión

1. ¿Qué cambia en el diseño si se asume que una credencial será filtrada alguna vez?
2. ¿Por qué una vulnerabilidad crítica en una dependencia no implica siempre el mismo riesgo?
3. ¿Qué dato dejaría de almacenar para reducir simultáneamente costo y exposición?

Capítulo 11

Desarrollo asistido por inteligencia artificial

Objetivos del capítulo

El estudiante podrá usar modelos generativos como colaboradores no confiables, diseñar contexto y criterios de aceptación, operar un modelo local cuando sea viable, evaluar código generado y mantener trazabilidad, privacidad y responsabilidad humana.

11.1. Qué cambia y qué permanece

Un modelo de lenguaje produce continuaciones probables a partir de instrucciones y contexto. Puede resumir, proponer código, generar pruebas, explicar errores y transformar artefactos con gran velocidad. No conoce automáticamente el propósito local, puede inventar APIs, reproducir vulnerabilidades y expresar certeza sin evidencia. Por eso la unidad de trabajo deja de ser “pedir código” y pasa a ser “formular, generar, inspeccionar, ejecutar y decidir”.

Permanece la responsabilidad profesional. Requisitos, arquitectura, seguridad, licencias y aceptación no se delegan al modelo. La fluidez del resultado no es una validación.

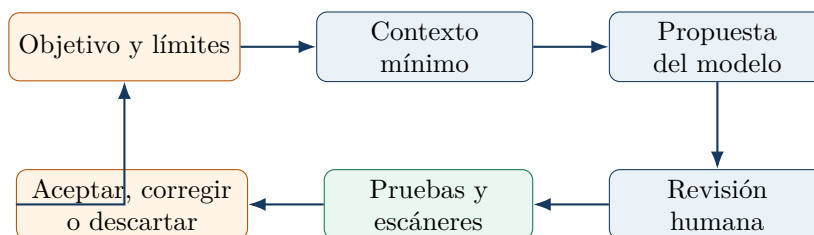


Figura 11.1: Bucle de asistencia con decisión y evidencia humanas.

11.2. Anatomía de una solicitud útil

Una solicitud técnica debe declarar resultado, contexto relevante, restricciones, formato y criterios de aceptación. Incluir todo el repositorio eleva costo y exposición; omitir contratos induce invenciones. Se seleccionan interfaces, errores, convenciones y pruebas cercanas.

Uso responsable de IA

Ejemplo: “En Python 3.13 y FastAPI, complete únicamente la función `crear_reserva`. Respete la interfaz adjunta, no añada dependencias, preserve códigos 201/409 y no registre datos personales. Primero enumere supuestos; luego entregue parche y pruebas pytest. Aceptación: suite existente, casos límite y análisis estático aprobados.”

Pedir que el modelo explique supuestos facilita revisión, pero la explicación también puede ser incorrecta. La verificación decisiva es ejecutar y contrastar con fuentes y contratos. Dividir una tarea limita el radio del error y mejora la posibilidad de atribuir cambios.

11.3. Usos y controles

Cuadro 11.1: Usos de IA y evidencia mínima.

Uso	Riesgo característico	Validación mínima
Generar código	API inexistente, caso límite, licencia incierta	Compilar, probar, revisar dependencia y procedencia.
Generar pruebas	Repetir el mismo error del código	Derivar casos desde requisitos y mutar una regla.
Refactorizar	Cambiar comportamiento silenciosamente	Pruebas caracterizadoras y comparación de contrato.
Documentar	Afirmar capacidades no implementadas	Ejecutar cada comando y enlazar evidencia.
Revisar seguridad	Falso negativo o consejo obsoleto	Modelo de amenazas, herramientas y revisión especializada.
Agente autónomo	Cambios amplios o acciones externas	Entorno aislado, permisos mínimos, límites y aprobación.

11.4. Agentes y permisos

Un agente combina modelo, instrucciones, memoria y herramientas en un ciclo. El riesgo no depende solo del texto que genera, sino de lo que puede leer y modificar. Se aplica mínimo privilegio: repositorio de práctica, secretos ausentes, comandos permitidos, presupuesto de pasos, registro de acciones y aprobación para publicar, desplegar o borrar.

El contenido recuperado desde documentos, incidencias o páginas puede incluir instrucciones maliciosas. Debe tratarse como datos, no como autoridad. Las reglas del curso y del sistema tienen precedencia; una sugerencia encontrada nunca autoriza extraer secretos ni ampliar permisos.

11.5. Modelos locales y herramientas libres

Ollama facilita ejecutar determinados modelos en una estación local; Continue es una interfaz de asistencia de código con código abierto que puede conectarse a modelos locales. “Local” mejora control de tránsito de datos, pero no elimina riesgos de licencia, telemetría, vulnerabilidades ni acceso al repositorio. El modelo concreto puede tener una licencia distinta a la herramienta que lo ejecuta: revísela antes de uso institucional o comercial.

Instalación requerida

Instale Ollama desde <https://ollama.com/download> y compruebe `ollama -version`. Instale la extensión Continue desde el mercado de su editor o siga <https://docs.continue.dev/guides/ollama-guide>. Descargue un modelo de código cuyo tamaño y licencia sean compatibles con el equipo:

```
1 ollama list
2 ollama pull qwen2.5-coder:7b
3 ollama run qwen2.5-coder:7b
```

El nombre es un ejemplo reproducible a la fecha de edición, no una recomendación permanente. Verifique catálogo, licencia y hash actuales. Un modelo de 7 mil millones de parámetros cuantizado suele requerir varios GiB de memoria; si el equipo no lo soporta, use el servicio institucional autorizado o realice el ejercicio por parejas sin subir información restringida.

11.6. Protocolo de evaluación

Una comparación válida congela tarea, contexto, versión del modelo, parámetros, entorno y pruebas. Evalúa corrección, seguridad, mantenibilidad, tiempo total y cantidad de intervención humana. Una sola demostración exitosa no permite concluir superioridad.

Validación

Prepare cinco tareas pequeñas con pruebas ocultas al modelo. Ejecute una línea base humana y una asistida. Registre versión, solicitud, archivos compartidos, salida, ediciones, pruebas, escaneos, tiempo y resultado. Revise manualmente una muestra de afirmaciones. No use datos personales ni código cuya política prohíba compartir.

11.7. Riesgos y gobernanza

El perfil de IA generativa del NIST organiza riesgos y acciones de gestión; la guía de UNESCO enfatiza un enfoque centrado en las personas para educación e investigación. En el aula se aplican mediante transparencia, alfabetización, protección de datos, acceso equitativo y evaluación auténtica.

Uso responsable de IA

Toda entrega declara herramienta y modelo, propósito, fragmentos afectados y validación humana. Se conserva un registro de solicitudes relevantes, no cadenas de pensamiento privadas. Está prohibido ingresar secretos, datos personales o material restringido. El estudiante debe explicar y modificar su solución sin asistencia. La calificación se basa en decisiones y evidencia, no en ocultar o maximizar uso de IA.

Los principales fallos son alucinación, código inseguro, sesgo, exposición de datos, dependencia de proveedor y pérdida de comprensión. Se enfrentan con fuentes, suites independientes, escaneo, minimización de contexto, alternativas interoperables, documentación y defensa oral.

Ejercicio propuesto

Laboratorio 11.1 — experimento controlado. Con Ollama y Continue, solicite implementar una regla y pruebas de LibreReserva. Compare con una solución sin asistente bajo el protocolo anterior. Entregue registro, diferencias, defectos encontrados y conclusión limitada por la muestra.

Laboratorio 11.2 — agente confinado. Configure un agente solo sobre una copia del proyecto, sin credenciales ni red externa si la plataforma lo permite. Autorícelo a proponer un refactor, no a confirmar ni desplegar. Revise el parche, ejecute validaciones y catalogue acciones que exigirían aprobación.

11.8. Preguntas de revisión

1. ¿Qué evidencia falsaría la afirmación de que la IA mejoró la productividad?
2. ¿Qué parte del repositorio no necesita ver el modelo para resolver una tarea concreta?
3. ¿Cómo se mantiene autonomía intelectual cuando la generación es casi instantánea?

Capítulo 12

Proyecto integrador: LibreReserva operable

Objetivos del capítulo

El estudiante integrará arquitectura, código, entrega, operación, seguridad e IA responsable en un producto pequeño, defendible y reproducible. El objetivo no es acumular tecnologías, sino demostrar decisiones proporcionadas mediante evidencia.

12.1. Problema y restricciones

LibreReserva permite publicar recursos institucionales, consultar disponibilidad, reservar, cancelar y auditar cambios. El equipo debe construir un incremento operable para dos perfiles: usuario y administrador. Se asumen datos sintéticos, presupuesto de infraestructura nulo y ejecución local con software libre.

Restricciones iniciales: API HTTP documentada; persistencia PostgreSQL; ejecución en contenedor OCI sin privilegio; automatización en Forgejo; despliegue local reproducible; telemetría OpenTelemetry; modelado de amenazas; registro de uso de IA. El equipo puede sustituir una herramienta por otra libre si mantiene la capacidad y justifica la decisión en un ADR.

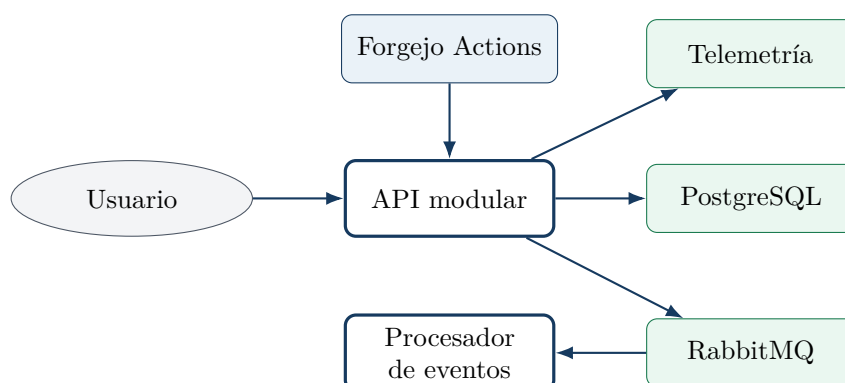


Figura 12.1: Arquitectura de referencia, deliberadamente sustituible mediante ADR.

12.2. Resultados de aprendizaje y evidencias

Cuadro 12.1: Matriz de trazabilidad del proyecto.

RA	Capacidad	Evidencia obligatoria
RA1	Analizar problema y restricciones	Mapa de actores, alcance, supuestos y riesgos.
RA2	Priorizar atributos de calidad	Tres escenarios medibles y tabla de compromisos.
RA3	Comunicar arquitectura	C4 de contexto/contenedores, UML selectivo y ADR.
RA4	Diseñar modularidad	Límites, puertos, dependencias y prueba de arquitectura.
RA5	Integrar datos y servicios	Contrato API, migración, idempotencia y prueba de integración.
RA6	Construir y validar	Incremento vertical, suite y reporte de calidad.
RA7	Entregar y operar	Canalización, imagen, manifiestos y despliegue reproducible.
RA8	Observar confiabilidad	SLI/SLO, panel, trazas y experimento de fallo.
RA9	Diseñar seguridad	Modelo de amenazas, SBOM, escaneos y triage.
RA10	Usar IA responsablemente	Registro, evaluación comparada, revisión y defensa.

12.3. Iteraciones

1. **Descubrimiento y arquitectura.** Problema, restricciones, escenarios de calidad, C4, modelo de dominio y dos ADR. Validación: revisión cruzada y prototipo de riesgo.
2. **Incremento ejecutable.** Crear y consultar reservas con persistencia, migración, manejo de conflictos y pruebas. Validación: contrato, cobertura razonada y demostración desde clon limpio.
3. **Entrega segura.** Forgejo Actions, imagen OCI, SBOM, secretos, escaneos y despliegue kind. Validación: digest trazable y puerta que rechaza un cambio deliberadamente defectuoso.
4. **Operación y resiliencia.** Telemetría, SLO, panel, alerta y experimento controlado. Validación: diagnóstico con métrica, traza y registro.
5. **Evaluación de IA y cierre.** Experimento asistido, deuda, costo, riesgos pendientes, manual reproducible y defensa individual.

Cada iteración termina con una versión demostrable. La retroalimentación modifica el producto y, cuando corresponde, el ADR o el modelo de amenazas. No se aceptan diagramas que describan un sistema distinto del entregado.

12.4. Definición de terminado

Una historia está terminada cuando: criterios de aceptación aprobados; revisión de otra persona; pruebas relevantes automatizadas; contrato y documentación actualizados; telemetría suficiente; amenazas examinadas; ningún secreto confirmado; canalización verde; imagen identificada por digest; y autoría/asistencia de IA declarada. Una excepción debe registrarse como deuda con riesgo, responsable y fecha.

12.5. Rúbrica

Cuadro 12.2: Rúbrica analítica del proyecto final.

Dimensión	Peso	Desempeño esperado
Problema y decisiones arquitectónicas	20 %	Restricciones explícitas, alternativas reales, escenarios medibles y ADR coherentes.
Construcción y pruebas	20 %	Incremento correcto, modular, legible y probado por riesgo; datos reproducibles.
Entrega y operación	15 %	Canalización confiable, artefacto trazable, despliegue y recuperación demostrados.
Observabilidad y confiabilidad	15 %	SLI/SLO defendible, señales correlacionadas y experimento que sustenta una conclusión.
Seguridad, privacidad y suministro	15 %	Amenazas priorizadas, controles probados, secretos protegidos, SBOM y triage.
IA responsable y trazabilidad	10 %	Uso declarado, contexto protegido, evaluación independiente y comprensión demostrada.
Comunicación y reproducibilidad	5 %	Documentación concisa, fuentes, licencias y ejecución desde entorno limpio.

Una falla crítica de integridad —credenciales reales, datos personales sin autorización, resultados fabricados o atribución deliberadamente falsa— se gestiona según el reglamento institucional y no se compensa con otros porcentajes.

12.6. Validación final

Validación

Un evaluador parte de un clon limpio y sigue el README. Debe poder: crear entorno; ejecutar pruebas; construir imagen; generar SBOM; desplegar; realizar una reserva; observar su traza; ejecutar un caso adverso autorizado y relacionar la versión desplegada con una confirmación. Todo comando dependiente del sistema indica plataforma y versión.

12.7. Defensa

La demostración dura quince minutos: problema y decisión principal; recorrido funcional; fallo inducido y diagnóstico; control de seguridad; resultado del experimento con IA; deuda reconocida. Luego cada integrante responde sobre una parte seleccionada al azar y realiza un cambio pequeño. Esto evalúa comprensión compartida y reduce la utilidad de entregas que el equipo no puede explicar.

Ejercicio propuesto

Entrega final. Repositorio, etiqueta de versión, digest de imagen, paquete de evidencias y breve informe arquitectónico. Incluya licencia del proyecto, atribuciones de componentes, instrucciones de instalación, matriz RA-evidencia y registro de decisiones. No entregue dependencias descargadas, modelos de IA ni secretos dentro del repositorio.

12.8. Cierre

La arquitectura moderna es una práctica de decisiones comprobables. Las plataformas, incluida la IA generativa, cambian con rapidez; la capacidad durable consiste en formular restricciones, comparar alternativas, construir ciclos de evidencia y responder responsablemente por el sistema resultante.

Apéndice A

Guía de instalación y entorno reproducible

Este apéndice concentra los requisitos de los laboratorios. Las versiones cambian; por eso se registran la versión instalada y la fuente oficial, en lugar de copiar comandos de terceros sin revisión. La ruta recomendada es GNU/Linux. macOS funciona con una máquina de Podman y Windows con WSL2 más una distribución Linux.

A.1. Capas del entorno

Cuadro A.1: Herramientas por capacidad.

Capacidad	Herramienta libre	Uso en el curso
Código y automatización	Git, Make, Python	Versionado, tareas repetibles y aplicación.
Editor y diagramas	VSCodium, PlantUML, Graphviz	Código, C4/UML como texto y exportación.
Aplicación y datos	FastAPI, PostgreSQL, RabbitMQ	API, persistencia y eventos.
Contenedores	Podman, Buildah	Construcción y ejecución OCI sin demonio central.
Forja y CI	Forgejo, Forgejo Runner	Revisión, incidencias y canalizaciones.
Orquestación	kubectrl, kind, Kubernetes	Despliegue local y reconciliación.
Observabilidad	OpenTelemetry, Prometheus, Grafana, Jaeger	Métricas, paneles y trazas.
Seguridad	Gitleaks, Syft, Trivy, OWASP ZAP	Secretos, SBOM, vulnerabilidades y DAST.
IA local	Ollama, Continue	Generación asistida bajo control del entorno.
Documento	TeX Live	Compilación de este libro y entregas técnicas.

A.2. Perfil mínimo y perfil ampliado

El perfil mínimo requiere procesador de 64 bits, 8 GiB de RAM y unos 20 GiB libres para repositorios, imágenes y datos. Permite ejecutar la API, base de datos y pruebas por separado. El perfil ampliado, con 16 GiB o más, permite kind, la pila de observabilidad y un modelo local pequeño. Estas cifras son orientativas: la arquitectura, la cuantización del modelo y los servicios simultáneos cambian el consumo.

Los laboratorios se pueden distribuir entre integrantes o ejecutar por etapas. La carencia de GPU no impide el curso; la IA local funciona más lentamente en CPU y puede reemplazarse por una instancia institucional autorizada. Ninguna evaluación exige pagar una suscripción.

A.3. GNU/Linux Debian o Ubuntu

Revise cada paquete antes de aceptar la instalación. Los nombres varían entre versiones de distribución.

```
1 sudo apt update
2 sudo apt install git make python3 python3-venv python3-pip \
3     podman podman-compose plantuml graphviz curl jq
4 git --version
5 python3 --version
6 podman --version
7 plantuml -version
```

Las herramientas que no estén en el repositorio se instalan desde su publicación oficial, comprobando firma o suma cuando se proporcione. No canalice directamente un script remoto al intérprete en un equipo institucional sin descargarlo, inspeccionarlo y verificar su procedencia.

A.4. macOS

Homebrew es software libre, pero su instalación es una decisión previa del usuario. Si ya está autorizado:

```
1 brew install git make python podman podman-compose plantuml graphviz jq
2 podman machine init
3 podman machine start
4 podman info
```

La VM de Podman necesita CPU, memoria y disco. Ajuste esos recursos al crearla si va a ejecutar kind u observabilidad. Las rutas montadas deben estar compartidas con la máquina.

A.5. Windows con WSL2

Active WSL2 mediante la documentación oficial de Microsoft e instale Ubuntu o Debian. Después siga la sección GNU/Linux dentro de la terminal WSL. Mantenga el repositorio en el sistema de archivos Linux, por ejemplo `/home/usuario/proyecto`, para evitar problemas de permisos y rendimiento. VSCodium puede conectarse a WSL mediante extensiones compatibles; también puede editar desde herramientas Linux.

Podman puede instalarse dentro de WSL o como Podman Desktop. Elija una ruta y documente dónde se ejecuta el motor para no confundir `localhost`, volúmenes y redes.

A.6. Entorno Python reproducible

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 python -m pip install --upgrade pip
4 python -m pip install -r requirements-dev.txt
5 python -m pip freeze
```

El archivo bloqueado debe registrar versiones y, en una entrega madura, hashes. Actualizar dependencias es una tarea explícita: se revisan cambios, licencia, vulnerabilidades y pruebas antes de confirmar el nuevo bloqueo.

A.7. Comprobación integral

```
1 git --version
2 python --version
3 podman info
4 podman run --rm docker.io/library/alpine:3.22 echo contenedor-ok
5 kubectrl version --client
6 kind version
7 trivy --version
8 syft version
9 gitleaks version
10 ollama --version
```

No todos los comandos son necesarios desde el primer capítulo. Un resultado “comando no encontrado” indica una instalación pendiente, no que deba instalarse sin revisar. Registre en `docs/entorno.md`: sistema, arquitectura, RAM, versiones, fecha y desviaciones.

A.8. Diagnóstico seguro

1. Lea el mensaje completo y determine si el fallo es versión, permiso, red, puerto o recurso.
2. Consulte la documentación oficial de la versión instalada.
3. Reduzca el caso: compruebe primero una herramienta, luego su integración.
4. No resuelva permisos usando `sudo` de forma indiscriminada ni cambie a usuario `root` dentro de la imagen.
5. No desactive TLS, firmas o escaneos para “hacer funcionar” una descarga.
6. Registre causa, corrección y prueba de que el arreglo funciona.

Validación

La instalación está terminada cuando un segundo integrante, usando `docs/entorno.md`, puede crear el entorno, ejecutar pruebas y levantar el servicio sin recibir credenciales personales. Una captura no reemplaza el comando reproducible ni su salida esperada.

Apéndice B

Plantillas de trabajo y evaluación

Estas plantillas reducen trabajo mecánico y hacen comparables las evidencias. Deben adaptarse; completar campos sin una decisión real no aporta valor.

B.1. Registro de decisión arquitectónica

Código B.1: docs/adr/NNNN-titulo.md

```
1 # NNNN. Título de la decision
2 Fecha: AAAA-MM-DD
3 Estado: propuesta | aceptada | reemplazada
4
5 ## Contexto
6 Problema, restricciones y fuerzas en tension.
7
8 ## Opciones consideradas
9 - Opcion A: beneficios, costos y riesgos.
10 - Opcion B: beneficios, costos y riesgos.
11
12 ## Decision
13 Opcion elegida y razon vinculada a escenarios de calidad.
14
15 ## Consecuencias
16 Efectos positivos, negativos, deuda y condiciones para revisarla.
17
18 ## Evidencia
19 Prueba, experimento, metrica o fuente que sustenta la decision.
```

B.2. Escenario de atributo de calidad

Fuente del estímulo

Estímulo

Entorno

Artefacto afectado

Respuesta esperada

Medida verificable

Método y fecha de
prueba

B.3. Registro de amenaza

Identificador y cate-
goría

Activo y límite de
confianza

Actor y precondi-
ción

Escenario de abuso

Probabilidad / im-
pacto

Control preventivo /
detectivo

Prueba del control

Riesgo residual, res-
ponsable, fecha

B.4. Registro de uso de IA

Fecha, integrante y tarea
Herramienta, modelo y versión
Datos o archivos compartidos
Solicitud o referencia al registro
Salida usada y archivos afectados
Ediciones humanas
Pruebas, escaneos y fuentes
Decisión: aceptar, corregir, descartar

No se exige registrar razonamiento privado ni conversaciones irrelevantes. Sí se registra suficiente información para comprender procedencia, alcance y verificación.

B.5. Informe de experimento

1. **Pregunta e hipótesis:** afirmación falsable antes de medir.
2. **Variables:** factor cambiado, respuestas medidas y controles.
3. **Entorno:** hardware, software, versiones, datos y configuración.
4. **Procedimiento:** pasos y repeticiones reproducibles.
5. **Resultados:** datos crudos, resumen y gráfica con unidad.
6. **Amenazas a la validez:** sesgos, muestra, calentamiento y ruido.
7. **Conclusión:** limitada por la evidencia; no generalizar más allá.

B.6. Lista de revisión de código

- El cambio corresponde a un requisito y tiene alcance comprensible.
- Los nombres expresan dominio; las dependencias respetan límites.
- Errores, tiempo, concurrencia e idempotencia se consideran cuando aplican.
- Las pruebas cubren comportamiento, casos límite y fallo relevante.
- No hay secretos, datos personales, registro excesivo ni privilegios innecesarios.
- Contratos, migraciones, telemetría y documentación permanecen coherentes.

- Código generado por IA fue declarado, revisado y validado.
- Licencias y atribuciones de nuevas dependencias fueron comprobadas.

B.7. Paquete de evidencia

Una entrega final puede organizarse así:

```
1 evidencias/  
2   arquitectura/      # C4, UML, ADR y escenarios  
3   calidad/           # pruebas, cobertura y experimentos  
4   entrega/           # ejecuciones CI, digest y manifiestos  
5   observabilidad/    # consultas, paneles y trazas  
6   seguridad/         # amenazas, SBOM, escaneos y triage  
7   ia/                # registro y evaluacion comparada  
8   matriz-ra.md       # enlace entre resultados y archivos
```

Los archivos de resultados automáticos deben poder regenerarse. Si son grandes, se conserva una síntesis y la instrucción exacta; no se llena el repositorio con imágenes de contenedor, modelos o bases de datos.

Referencias y recursos técnicos

Las versiones y páginas se verificaron durante la edición de 2026. En prácticas posteriores debe consultarse la documentación correspondiente a la versión instalada.

- [1] Mark Richards y Neal Ford. *Fundamentals of Software Architecture*, 2.^a ed. O'Reilly Media, 2025. <https://www.oreilly.com/library/view/fundamentals-of-software/9781098175504/>
- [2] IEEE Computer Society. *Guide to the Software Engineering Body of Knowledge*. <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [3] Simon Brown. *The C4 model for visualising software architecture*. <https://c4model.com/>
- [4] NIST. *Secure Software Development Framework (SSDF), SP 800-218*. <https://csrc.nist.gov/pubs/sp/800/218/final>
- [5] NIST. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. Página oficial del perfil de IA generativa
- [6] UNESCO. *Guía para el uso de IA generativa en educación e investigación*. <https://www.unesco.org/es/articles/guia-para-el-uso-de-ia-generativa-en-educacion-e-investigacion>
- [7] CISA y FBI. *Product Security Bad Practices*. Guía oficial de CISA y FBI
- [8] DORA. *State of AI-assisted Software Development 2025*. <https://dora.dev/research/2025/dora-report/>
- [9] FastAPI. *Documentación oficial*. <https://fastapi.tiangolo.com/>
- [10] Podman. *Installation Instructions*. <https://podman.io/docs/installation>
- [11] Forgejo. *Documentation y Forgejo Actions*. <https://forgejo.org/docs/latest/>
- [12] Kubernetes SIGs. *kind Quick Start*. <https://kind.sigs.k8s.io/docs/user/quick-start/>
- [13] OpenTelemetry. *Documentation y Demo*. <https://opentelemetry.io/docs/>
- [14] Prometheus Authors. *Installation*. <https://prometheus.io/docs/prometheus/latest/installation/>
- [15] Grafana Labs. *Configure a Grafana Docker image*. <https://grafana.com/docs/grafana/latest/setup-grafana/configure-docker/>
- [16] Aqua Security. *Trivy Installation*. <https://www.trivy.dev/docs/latest/getting-started/installation/>

- [17] Anchore. *Syft: SBOM generation tool*. <https://github.com/anchore/syft>
- [18] Gitleaks. *Detect secrets in source code*. <https://github.com/gitleaks/gitleaks>
- [19] OWASP ZAP. *Docker documentation*. <https://www.zaproxy.org/docs/docker/about/>
- [20] Continue. *Using Ollama with Continue*. <https://docs.continue.dev/guides/ollama-guide>
- [21] Ollama. *Download and installation*. <https://ollama.com/download>